

When a Learned Path Integrator Makes a Map: Controlled Measurements on MapFormer (short version)

Prashant Rangarajan

Short version of the results report of 13 September 2026

Abstract

MapFormer accumulates a content-dependent, signed phase and applies it as a rotation in attention. We measure what that accumulated “where” buys when the observation map is redrawn at test, so that every evaluation measures transfer to a new instance of a known structure. On the paper’s task under a converged recipe, path integration is the larger factor on raw accuracy at every length (+0.243 at training length, +0.359 at eight times it, $n = 8$). It pays under three conditions. Input must be integrable: under turn-and-forward actions the advantage over an index code falls from +0.438 to +0.050, and recording realised displacement restores it to +0.488. Increments must cancel: on the torus at eight times the training length, forcing them non-negative costs 0.363 raw and 0.280 at matched loss (loss-matched verdicts depend on the pool), extending a known result to navigation. Solutions must be findable: single-origin MapEM represents a varying-offset recall task exactly yet trails MapWM by 0.375 from scratch at a shared budget. Explicit correction and gating add no measurable accuracy at training length, pooling’s only detectable gain is ceiling-level (its larger gains are directional); recursion adds accuracy where the base model trains unreliably (on Match-Query, in an unpaired analysis). Transfer across a change of structure is not tested.

1 Introduction

A model that knows *where* it is, separately from *what* it sees there, should be able to carry that knowledge into a place it has never observed. This is the claim of the Tolman–Eichenbaum Machine (Whittington et al., 2020): structural knowledge is invariant across environments, sensory content is specific to one, memory stores their conjunction, and the factorisation is what buys transfer. This report asks when a transformer’s positional mechanism can serve as that structural code.

MapFormer (Rambaud et al., 2025) carries the factorisation into attention. Each token, action or observation, contributes a learned increment to a phase; the phase is accumulated by a prefix sum and applied as a rotation. In MapFormer-WM (MapWM) the rotation acts on content-derived queries and keys; in MapFormer-EM (MapEM) it acts on two learned position vectors, producing a position kernel that multiplies a content score. Except for two appendix baselines of the full report, every evaluation here redraws the observation map from a held-out seed, so only structure can carry over: every accuracy below measures transfer to a new instance of a known structure. None of it measures transfer across a change of structure (topology or action space), the stronger form of the TEM claim.

What is already known. The two-slot taxonomy of rotary logits is due to GRAPE (Zhang et al., 2026) and Puranik (2026); Vetcha (2026) states the two-slot decomposition independently. Content-dependent rotation is published by Mamba-3 (Lahoti et al., 2026) and Selective RoPE (Movahedi et al., 2026). That non-negative state transitions cannot track parity and that negative values restore it is due to Sarrof et al. (2024) and Grazzi et al. (2025); Selective RoPE’s Section 4.2 carries it into rotations in attention. That a fixed index code cannot count contextually and a content-gated position can is CoPE’s claim (Golovneva et al., 2024). MapFormer’s two variants follow Whittington et al. (2025). That a weight-shared block applied repeatedly can substitute for depth is the premise of Mixture-of-Recursions (Bae et al., 2025).

The argument. On tasks where the observation map is redrawn at test, the transferable part of MapFormer is its accumulated, content-driven position phase. That phase pays under three conditions, each supported by an intervention on its own task; that rank acts through conditioning is descriptive. The input must make a token’s displacement fixed; the increment must be signed and well-conditioned so that it can cancel; and training must find the solution. The conditions are not claimed to be sufficient, they are not an exhaustive list, and no single task shows all three binding at once. The report makes five claims:

1. The accumulated phase carries transfer (Section 3).
2. The environment must make displacement a function of the token (Section 4).
3. The increment must cancel, and cancel cleanly (Section 5).
4. A shared kernel is representable but harder to find (Section 6).
5. What added machinery buys: only recursion measurably adds more than a ceiling-level amount of accuracy at training length on the tasks tested (Section 7).

2 Setup

The accumulator. MapFormer maps every token x_t (action or observation) through a rank- r bottleneck to a phase increment and accumulates it:

$$\Delta_t = W_{\text{out}} W_{\text{in}} x_t \in \mathbb{R}^{n_b}, \quad S_t = \sum_{u \leq t} \Delta_u, \quad \theta_t = \omega \odot S_t, \quad (1)$$

with n_b frequency blocks and learned angular velocities ω . The paper uses $r = 2$. Index RoPE (Su et al., 2021) is the special case $\Delta_t \equiv 1$. The attention between positions t and s depends on $S_t - S_s$, the interval sum of the increments between them.

WM and EM. MapEM’s attention is $\text{softmax}(A_X \odot A_P)$, a Hadamard product of a content score A_X and a position score built from two learned vectors,

$$A_P[t, s] = q_0^\top R(\theta_s - \theta_t) k_0 = \sum_i a_i \cos(\omega_i (S_t - S_s) + \phi_i) =: \kappa(\Delta S). \quad (2)$$

One kernel κ serves every query–key pair. The paper uses separate q_0 and k_0 (App. A.7 of v4); the ablation used throughout is a single shared origin p_0 ($q_0 = k_0 = p_0$, so $\phi_i = 0$). The Hadamard product is exactly TEM’s conjunction, and the sign of κ is a gauge: $A_X \odot (-\kappa) = (-A_X) \odot \kappa$. MapWM rotates content-derived Q, K : $Q_t^\top R(\theta_s - \theta_t) K_s = \sum_b |Q_b| |K_b| \cos(\omega_b \Delta S + \phi_b(q, k))$, so amplitudes and phases are set per block *and per pair* by content. MapWM is therefore not an additive combination of a content and a position score.

Tasks. Table 1. Every task evaluates on held-out episodes and, where there is an observation map, on a map drawn from a held-out seed. A task is used only after it passes shortcut gates run on the task code itself: n -gram predictors of orders 1–5 on the action or answer stream alone and task-specific trivial predictors must sit at the chance or floor rate.

Statistics. Every contrast is a difference paired by seed. We call it *detectable* when it exceeds the smallest effect the design could reliably detect, and *unmeasured* otherwise (which is not evidence of zero); a negative result is *powered* when the design could have detected the effect it dismisses. The full report gives every effect with its minimum detectable effect and seed counts. Results at $n \leq 3$ are exploratory. Compared arms were trained in one batch with the same code; most batches from September 2026 were pre-registered, verdicts are reported as registered, and post-hoc analyses are labelled. Where accuracy tracks final training loss, contrasts are also given *loss-matched* (at equal final loss, by pooled regression), a control whose verdict depends on the pool. The *paper* recipe is 16 epochs with linear decay from step one at lr 3×10^{-4} ; the converged *torus* recipe is 300 epochs, 5% warmup and cosine, lr 10^{-3} . Match-Query does not reproduce across batches or same-seed retrains, so its seed-paired contrasts are unreliable.

Table 1: Tasks. *Chance / floor*: chance = one over the number of answers; “floor” is a measured no-model baseline on the scored events; multiple floors are for MiniGrid at $T = 128/512/1024$ and for MiniWorld at 16/64/256 observation types. *Train / eval length*: sequence length in training / at evaluation. Reading: every task evaluates on held-out episodes and, where there is an observation map, on a map drawn from a held-out seed.

task	tests	chance / floor	train / eval length
Paper task (torus 64×64 , 16 observation types, loss at revisits)	in-context map, revisit prediction	blank floor 0.506	$T = 128 / 128$, 512, 1024
Knob torus (turn/forward actions)	whether a fixed per-token increment can name a displacement	floor 0.508	$T = 128 / 128$
Match-Query 128 ²	blind continuation: observations withheld in the query phase	chance 0.0625	explore 512, query 256
Parity	a signed prefix sum as a register	chance 0.5	$L = 16 / 16$ –256
Recency (k -back, uncounted filler)	contextual position; varying retrieval offset	chance 0.0625; most-recent floor 0.0771	$T = 1024 / 1024$, 2048
MiniGrid DoorKey-16x16	an external benchmark, commanded or allocentric actions	floors 0.635 / 0.536 / 0.490	$T = 128 / 128$, 512, 1024
MiniWorld OneRoom (fresh map)	map extent and aliasing	non-blank marginal 0.077 / 0.031 / 0.013	$T = 512 / 512$

3 Claim 1: the accumulated phase carries transfer

On four tasks with a redrawn map or held-out episodes, an index code falls well short of a path-integrated code, and on the paper task it sits near its floor under the paper’s recipe; on MapFormer’s own task path integration is the larger factor on raw accuracy at every length.

The paper task. Table 2 crosses the encoding (RoPE or PoPE) with the source of the angle (the index, or MapFormer’s path integral), with $r = 4$ path-integrated arms added; all three registered predictions held. The position main effect at $r = 2$ is $+0.243$ at training length and $+0.359$ at eight times it, both detectable. The encoding main effect is -0.049 and $+0.189$ at the same lengths, both detectable; at training length it is inside the registered 0.05 bound by 0.001, and it is negative mostly because PoPE lowers the index code. Under the paper’s recipe, in a different batch and evaluator, index models sit just above the 0.506 blank floor and the position main effect at training length is $+0.461$ (its detectability was not recorded), with the encoding main effect $+0.003$, unmeasured; both index arms score higher under the converged recipe (RoPE $0.530 \rightarrow 0.805$, PoPE-Flat $0.509 \rightarrow 0.679$). The maps are used: destroying the context on trained models ($n = 3$, exploratory by n), **Vanilla** falls from 0.9889 intact to 0.1783 with actions resampled from another episode, below the blank floor.

Why the position effect is not loss-matched. Accuracy tracks final training loss closely here ($r = -0.961$ at $T = 128$), which would normally call for comparing arms at matched loss, to separate a model that represents the task better from one that merely trained further. For position that comparison does not exist: index arms end training at loss 0.68–0.96 and path-integrated arms at 0.00–0.38, because an index code cannot fit the task. Matching on loss would remove the very difference being measured, so the position effect is reported on raw accuracy only. Whether path integration represents the map, rather than only fitting it, is shown by the interventions of Sections 4–6, not by this table. The encoding effect, whose arms do share a loss range, survives loss-matching at every length (PAPER2X2_RESULTS.md).

Other tasks. Match-Query withholds observations in the query phase, which closes the retrace route that lets an index model exceed the floor on the paper task. On the 128² map, with eight seeds per arm pooled

Table 2: Paper task under the converged torus recipe: accuracy predicting the observation at revisited cells of a held-out map, trained at $T = 128$, evaluated up to eight times that; 6 arms $\times$ 8 seeds in one batch, pre-registered. *Enc.*: RoPE = rotation only; PoPE = content-dependent magnitude plus rotation. *Position*: index = token index; path-int. = MapFormer’s running sum of learned per-token increments, bottleneck rank r . **Vanilla** = MapFormer-WM; RoPE = MapFormer with index rotation. Top rows: accuracy mean $\pm$ sd. Bottom rows: main effects on accuracy, each averaged over the other factor and paired by seed; *position* = path-integrated minus index; *encoding* = PoPE minus RoPE. Every main effect shown is detectable. Effects are on raw accuracy; why no loss-matched position effect is reported is explained in the text. Blank floor about 0.51 at $T = 128$. Reading: path integration is the larger factor at every length; the encoding effect is detectably negative at training length and positive beyond it.

arm	enc.	position	$T = 128$	$T = 512$	$T = 1024$
RoPE	RoPE	index	0.805 ± 0.012	0.470 ± 0.093	0.374 ± 0.135
PoPE-Flat	PoPE	index	0.679 ± 0.058	0.639 ± 0.046	0.607 ± 0.033
Vanilla	RoPE	path-int., $r = 2$	0.971 ± 0.043	0.915 ± 0.052	0.777 ± 0.094
MapPoPE-Flat	PoPE	path-int., $r = 2$	0.999 ± 0.001	0.974 ± 0.011	0.921 ± 0.023
Vanilla_r4	RoPE	path-int., $r = 4$	1.000 ± 0.000	0.985 ± 0.006	0.928 ± 0.012
MapPoPE_r4	PoPE	path-int., $r = 4$	1.000 ± 0.001	0.982 ± 0.018	0.941 ± 0.028
position, $r = 2$			+0.243	+0.390	+0.359
encoding, $r = 2$			−0.049	+0.114	+0.189
position, $r = 4$			+0.258	+0.429	+0.444
encoding, $r = 4$			−0.063	+0.083	+0.123

from three launches, a one-layer path-integrated model scores 0.456 ± 0.220 and a parameter-matched index model 0.108 ± 0.025 , against chance 0.0625; the paired estimate +0.348 is unreliable, because same-seed retrains on this task drift by 0.185 per seed, and no unpaired test of this contrast was run. On k -back recency, path-integrated minus index is +0.750 at $T = 1024$, detectable; index arms score 0.99–1.00 at $k = 1$ and 0.11–0.13 at $k = 64$. That a fixed index code cannot count contextually is CoPE’s published claim; this is a replication.

Full report: report.pdf, Sections 4–5 (reproduction of the paper’s Table 2 and figure claims, parity, the recipe objection).

4 Claim 2: the environment must make displacement a function of the token

MapFormer adds a fixed increment per token. Under turn-and-forward actions the displacement produced by “forward” depends on the accumulated heading, which no fixed increment can represent. On the torus the position effect falls from +0.438 to +0.050 under rotation-based actions, and recording the realised displacement instead restores it to +0.488 ($n = 8$). On MiniGrid with allocentric actions, path integration beats an index code only at $r = 4$ (+0.034, detectable). On MiniWorld the effect does not shrink as observation aliasing falls: in converged cells it is +0.178 at 32 cells per token ($n = 5$, 400 epochs) and +0.305 at 2 ($n = 3$, 800 epochs, exploratory).

Rotation and allocentric recording. Five properties differ between the torus and MiniGrid. Turning each on separately in the torus code singled out rotation-based actions, which were re-run at eight seeds (Table 3). Under the allocentric record the agent’s dynamics are byte-identical to the rotation condition; only the recorded action token changes, from the commanded turn or forward to the realised absolute displacement. The answer-stream n -gram gates are identical to three decimals under both records, and the index arm stays at the floor under both. The recoding supplies an integrable input, not the target: the result shows that a fixed per-token increment cannot perform the heading-to-displacement computation, not that a map can do without it. It requires the agent’s heading to be known. At 12 headings, with real-valued position, every

Table 3: Torus knob sweep: accuracy predicting the observation at revisited cells of a held-out map, at $T = 128$. Rotate = turn and forward actions (displacement depends on heading); allocentric record = same dynamics, each action token the realised displacement. Path-integrated = **Vanilla** (MapFormer-WM); index = **RoPE** (position = token index). Mean $\pm$ sd, $n = 8$ per arm; effect = difference of means; floor = measured no-model baseline. All arms trained 16 epochs; rotate and allocentric at 392 batches per epoch (matched supervised events), baseline and combined at 98. No uncertainty was recorded for the effects. The combined condition’s order-3 answer-stream gate reached 0.634 against a 0.536 marginal ($n = 3$ gate), so its effect is approximate. Reading: rotation removes most of the effect, and recording the realised displacement restores it.

condition	floor	path-integrated	index	effect
baseline (translation actions)	0.506	0.967 ± 0.039	0.529 ± 0.044	+0.438
rotate (turn / forward)	0.508	0.558 ± 0.026	0.508 ± 0.006	+0.050
rotate, allocentric record	0.508	0.996 ± 0.005	0.508 ± 0.006	+0.488
all five knobs combined	0.552	0.708 ± 0.068	0.785 ± 0.022	−0.076

path-integrated seed is above every index seed at every budget tried ($n = 3$ per budget); the direction is an existence statement, the magnitude exploratory.

MiniGrid. On DoorKey-16x16 with the environment’s own turn-and-forward actions, index arms come first (PoPE-Hier 0.955 ± 0.003 at $T = 1024$, with the paper’s MapWM-Flat last at 0.823 ± 0.088 ; $n = 8$, floor 0.490), the regime the knob sweep predicts; the main effects there have no recorded uncertainty and are directional. With actions recorded as realised grid displacement (one batch, $n = 8$), **Vanilla_r4** minus **RoPE** is +0.034 at $T = 1024$, detectable; at the paper’s $r = 2$ the contrast is -0.010 , unmeasured. The effect is small, on a benchmark where every arm is far above the floor; the claim is the sign conditioned on $r = 4$, not its size.

Caveats. The knob effects have no recorded uncertainty, and the allocentric arm was trained at matched supervised events, four times the baseline’s batches. On MiniWorld, the falsification of the aliasing account rests on two converged cells that differ in budget and n , and no test of their difference was run; the two arms’ final losses do not overlap at $n_{\text{obs}} = 64$ and 256, so that study measures “optimises better”, not “represents better at equal fit”.

Full report: report.pdf, Section 6 and Appendix E (heading budgets, MiniWorld convergence and map extent, Habitat).

5 Claim 3: the increment must cancel, and cancel cleanly

For the interval sum to measure net displacement, opposite actions must produce cancelling increments, and the basis they live in must be well conditioned. Forcing the increment non-negative costs MapWM 0.363 raw and 0.280 at matched loss on the torus at $T = 1024$, both detectable, and at matched loss leaves no measurable gain over an index code (raw, the monotone arm stays ahead of it, +0.213); Selective RoPE’s generator pays the same raw cost. The paper’s $r = 2$ bottleneck learns a skewed basis, and $r = 4$ adds +0.085 at $T = 1024$ for 384 parameters. For the two WM-type generators the constraint’s cost is large on a map task and small on a counting task.

Sign. Six arms, twelve seeds, one batch, torus recipe: a signed arm, **Abs_r4** ($|\Delta|$, the one-operation isolation), the non-negative parameterisations of GRAPE’s additive path and CARoPE (Veisi et al., 2025), a construction control, and index **RoPE**; all five MapFormer arms have 204,757 parameters. At matched loss, signed path integration beats the index code by +0.123 and +0.195 at $T = 512$ and 1024, both detectable, and the monotone arm beats it nowhere; on raw accuracy the monotone arm is detectably above the index code at every length, because **RoPE** trains to a much higher loss. At $T = 128$ the signed arm sits at 1.000 ± 0.000 , so the pre-registered training-length accuracy discriminator could not show a deficit, a design error recorded before any result was read; the training-length effect is in the loss, higher for every constrained arm on every seed. The opposition score $\|D(+a) + D(-a)\|/\|D\|$ is 0 when opposite actions cancel and 2 when they are identical: 0.125 and 0.106 for the signed arm, against 1.849/1.855 for **Abs** and 1.981/1.977 for **CARoPE** (seeds 0–2). Cancellation chooses what the accumulator measures: signed increments give net displacement, a map; monotone increments give path length, a clock. On recency (Table 4) the constraint’s raw cost is small

Table 4: Cost of the monotone constraint (increments forced non-negative, minus signed), $T = 1024$, paired by seed, $n = 12$. Torus = paper task (trained at $T = 128$); recency = return the k -th most recent symbol past uncounted filler (trained at $T = 1024$). The MapWM torus row is the stored sign ablation (six-arm loss-matching pool); all other rows are the pre-registered MONOTONE batch. Raw = difference in accuracy; loss-matched = compared at equal final training loss by pooled regression; unmeasured is not evidence of zero. Reading: for the two WM-type generators the cost is large on the map task and small on the counting task, while single-origin MapEM pays more than MapWM on recency, a difference that is unmeasured.

task	generator	raw	loss-matched	verdict
torus	MapWM ($r = 4$)	-0.363	-0.280	detectable
torus	Selective RoPE's (SRoPEGen)	-0.355	-0.188, eight-arm pool, exploratory	detectable
recency	MapWM ($r = 4$)	-0.060	-	unmeasured
recency	Selective RoPE's (SRoPEGen)	-0.068	-	detectable
recency	single-origin MapEM ($r = 4$)	-0.198	-0.043	raw detectable; loss-matched unmeasured
SRoPEGen cost - MapWM cost, torus raw (unpaired): +0.009, unmeasured				
SRoPEGen, torus (raw) - recency: -0.287, detectable				
MapEM cost - MapWM cost, recency: -0.138, unmeasured				

for both WM-type generators (unmeasured for MapWM, detectable for Selective RoPE's generator), but single-origin MapEM pays 0.198 raw; the registered verdict on that batch's MapEM question is unresolved. The sign axis and the parity theorem are due to Sarroff et al. (2024) and Grazzi et al. (2025), and were carried into content-dependent rotation in attention by Movahedi et al. (2026); what is new here is the navigation regime and the one-operation isolation at identical parameter count.

Rank. Only r varies across five arms, eight seeds each, one batch, torus recipe. Against $r = 2$, $r = 4$ adds +0.085 at $T = 1024$, detectable, and +0.038 at $T = 512$. The gain is a step: $r = 8$, 16 and 32 give +0.091, +0.079 and +0.095 at $T = 1024$. $r = 4$ also cuts the $T = 1024$ seed sd from 0.064 to 0.012, at a cost of 384 parameters; $r = 1$ and $r = 3$ were not run. What fails at $r = 2$ is the basis: opposite actions fail to cancel by half the action scale (opposition 0.4950 against 0.0922 at $r = 4$), and north and east are nearly parallel ($|\cos| = 0.7833$ against 0.1754). This is a description of the learned codes on eight seeds; the causal route from skew to accuracy is not tested. The result is conditional: on MapPoPE, $r = 4$ against $r = 2$ is +0.019 at $T = 1024$, unmeasured, so "use $r = 4$ " is a MapWM-family recommendation.

The main caveat. Loss-matched verdicts depend on the pool. The registered loss-matched form for Selective RoPE's generator was falsified as registered: its two-arm pool had non-overlapping final losses, so the fit absorbed the effect by construction and returned -0.056, unmeasured.

Full report: report.pdf, Section 7 and Appendices F, K (per-arm sign and recency tables, the counting gate, the accumulator exponent, packing geometry).

6 Claim 4: a shared kernel is representable but harder to find

MapEM factorises where from what completely: one position kernel is shared by every query-key pair. On map tasks no detectable difference from MapWM was found once its initialisation is fixed. On varying-offset recency, single-origin MapEM trails MapWM by 0.375, detectable. The gap is not in what the model can represent: the solution installed and frozen scores 1.000, and installed at training scale with a content leak closed it is held at 1.000. From scratch MapEM finds the solution per query token, in wrapped form. Per-pair position origins add +0.215 over single-origin MapEM at $n = 48$, split between pathway capacity (+0.124) and per-pair freedom (+0.091), and only the freedom changes the mechanism.

The gap. Whittington et al. (2025) predicts that episodic-memory models learn faster than working-memory models except on N-back. On varying- k recency (eight seeds per arm, one batch), MapWM reaches training loss below 0.5 on every seed and single-origin MapEM on none. Single-origin MapEM minus MapWM is -0.375 at $T = 1024$; the paper's separate-origin form minus MapWM is -0.137; both are detectable. MapWM at four times the budget was not run, so -0.375 is a comparison at a shared budget and says nothing about asymptotic ability. On allocentric MiniGrid, by contrast, single-origin MapEM minus MapWM (both $r = 4$) is +0.0035 at $T = 1024$, unmeasured (no detection threshold recorded).

Table 5: The existence and holdability ladder for single-origin MapEM on recency (retrieval accuracy, trained at $T = 1024$), $n = 8$ per arm (seeds 0–7). “Install” writes the rewind code into the position pathway, frozen or trainable; “scale” is the weight scale at which it is written; “leak closed” holds the content columns of W_{in} at zero. $T = 2048$ values are means. Reading: the solution is represented exactly, and held at eight times the scale with the leak closed.

arm	$T = 1024$	$T = 2048$
from scratch (VanillaEM.P0.r4)	0.600 ± 0.126	–
installed, frozen	1.000 ± 0.000	1.000 ± 0.000
installed at scale 1/64, trainable	0.642 ± 0.266	0.582
installed at scale 1/64, trainable, leak closed	0.784 ± 0.281	0.709
installed at scale 1/8 ($8\times$), trainable	0.941 ± 0.063	0.895
installed at $8\times$, trainable, leak closed	1.000 ± 0.000	0.991
MapWM (Vanilla.r4)	0.975 ± 0.072	–

Existence and holdability. A signed accumulator can do k -back without counting upward: if symbols carry $\Delta = (1, 0)$ and the query token for offset k carries $\Delta = (-(k - 1), 0)$, the prefix sum rewinds the count at the query. Installed in the position pathway and frozen, with the content branch learned from random initialisation, this scores 1.000 ± 0.000 at $T = 1024$ and $T = 2048$ (Table 5). Made trainable from an install written in small weights, the model falls back to 0.642; written at eight times the weight scale it holds 0.941, and with the content leak into Δ through W_{in} closed it scores 1.000 ± 0.000 , meeting the registered readout on every seed (the paired accuracy contrast, +0.059, is at ceiling and unmeasured).

Search. Because θ enters attention through cosines and sines, a block’s rewind only has to hold modulo $2\pi/\omega_i$. Read as a function rather than a linear slope (five from-scratch MapEM arms, 24 seeds each), 93–97% of solved large- k cells are retrieved through the query token’s own wrapped rewind, to the kernel’s peak when A_X is positive and its trough when it is negative, and a fraction 0.000 of failed cells carry a rewind (a description; the route table was not registered). With one shared k , single-origin MapEM finds a 63-symbol rewind on most seeds, all wrapped. Queries per token is the currency: with k drawn from a set of m offsets, at a fixed budget $m = 4$ minus $m = 64$ is +0.665, detectable, while at matched queries per token $m = 16$ at four times the epochs minus $m = 4$ is +0.045, unmeasured. Exposure was varied through the budget, so queries per token and optimiser steps per token are not separated.

Per-pair origins. **EMPair_r4** gives each token its own origin ($q_t^p = p_0 + W_{\text{out}}^q W_{\text{in}}^q x_t$, likewise for keys) and starts as single-origin MapEM exactly; the capacity control **EMPairConst_r4** uses the same pathway reading a learned constant, so the kernel stays shared. At $n = 48$ all three terms are detectable: total +0.215, pathway +0.124 and freedom +0.091; the freedom term replicates on the 40 fresh seeds alone (+0.089, detectable). At $n = 8$ the total was +0.280, so the first eight seeds overstated it, and **EMPair** minus MapWM was -0.095 , unmeasured: per-pair origins are not shown to close the gap to MapWM. On descriptive $n = 8$ readouts, the control keeps the per-token rewind route (0.948, against single-origin MapEM’s 0.964) and zero phase spread; only per-pair freedom abandons the route (0.189) and spreads the kernel phase across pairs (1.448).

The main caveat. Every recency contrast in this section is a contrast of fit, with $r(\text{loss}, \text{accuracy})$ between -0.936 and -0.987 ; the “search, not representation” reading rests on the existence and holding constructions, not on loss-matching. “Holds” is shown on one task, at $n = 8$, with the content columns of W_{in} pinned: a point in weight space that the unconstrained optimiser drifts away from.

Full report: report.pdf, Section 8 and Appendix G (kernel phase probe, map tasks, availability, monotone MapEM, phase freedom in the origins).

7 Claim 5: what added machinery buys

Of the additions tested (weight-shared recursion, an explicit token gate, Selective RoPE’s generator, a forget gate, hierarchical pooling, and explicit state correction), only recursion measurably adds more than a ceiling-level amount of accuracy at training length on the tasks tested, and recursion does so where the base model trains unreliably.

Table 6: Headline contrasts for added machinery. Each row comes from its own batch, task and recipe; rows are not comparable with each other. **Level15** = MapFormer-WM (**Vanilla**) plus a wrapped Kalman filter; **Gated_r4** = per-token sigmoid gate on the increment; hourglass = hierarchical pooling; loop = one shared layer applied four times. Estimates: accuracy difference, raw unless marked; loss-matched = at equal final training loss. Powered negative = the design could have detected the effect it dismisses; directional = consistent in sign but not detectable; take 1 / take 2 = two recipes. Reading: only recursion measurably adds more than a ceiling-level amount at training length, where the base model trains unreliably.

addition	task	contrast	estimate	reading
state correction	Match-Query, $p = 0.10$, take 1 / take 2	Level15 – Vanilla	+0.038 / +0.005, both unmeasured	no measurable benefit
state correction	clean torus, $T = 1024$, $n = 12$	Level15 – Vanilla	+0.129, detectable; +0.083 loss-matched, at the detection threshold	beyond training length only
token gate	torus, $T = 512$, $n = 8$	Gated_r4 – Vanilla_r4	+0.004	powered negative
hierarchy	parity, $L = 16$, $n = 16$	hourglass – flat	+0.012, detectable	ceiling-level
hierarchy	compositional motif, $T = 512$, $n = 8$	hourglass – flat	+0.151	directional
recursion	Match-Query 128 ²	loop $\times 4$ – one layer, path-int.	+0.346, unpaired	survives unpaired; not loss-matched
recursion	torus, $T = 128$, $n = 12$	looped – Vanilla	+0.052, detectable; +0.006 loss-matched, unmeasured	convergence

State correction. The project’s original extension was a wrapped invariant extended Kalman filter on the path integral (**Level15**). Its premise is drift in the integrated position together with observations that carry information about the true position; neither is present on the paper’s task, so stochastic transitions were added to Match-Query’s explore phase (at $p = 0.10$ the recorded path drifts by 13.05 cells from the true one, and the n -gram gates hold). The pre-registered prediction that **Level15** minus **Vanilla** grows with drift was not supported in either of two recipes (Table 6): no benefit is detectable at $p = 0.10$, in arms that are not converged, while the $p = 0$ contrasts are too imprecise to resolve one, so the slope itself is unmeasured. Destroying the query actions leaves about 45% of the $p = 0.10$ score in place, so both filter contrasts are differences of two numbers near a non-positional floor of about 0.15. On the clean torus the filter is detectable only beyond training length, no named component is load-bearing, and there is no evidence that it works by inference from observations.

Gate and hierarchy. **Gated_r4** multiplies MapFormer’s signed increment by a per-token sigmoid (CoPE’s selection applied to MapFormer’s direction, 258 parameters). It separates, passing 0.987 ± 0.003 on actions and 0.244 ± 0.043 on observations, a ratio of 4.16, and it does not help: on the torus at $T = 512$ the contrast is a powered negative against any gain the size of rank’s +0.038 there. This suggests that MapFormer’s bottleneck already separates the two token types. On a motif task built so that a pooled summary is a sufficient statistic, a parameter-identical hourglass minus its flat scaffold is +0.136 at $T = 256$ and +0.151 at $T = 512$: directional at $n = 8$, unmeasured; on the same task the recipe change alone is +0.160, detectable. Hierarchy’s only detectable gain is the ceiling-level +0.012 on parity, from 0.988 to 1.000.

Recursion. On Match-Query, where seed-paired contrasts are unreliable, a four-pass loop on a path-integrated model survives an unpaired analysis (+0.346, looped arm pooled over two batches, $n = 16$ against 8); the paired interaction with path integration, +0.315, detectable, has not been re-checked unpaired. Unpaired, the loop against three real layers is +0.032: not distinguishable from them, at a third of the parameters. With $r = 4$ the looped model scores 0.986 ± 0.020 in its batch. On the torus the loop’s training-length gain is +0.052 raw but +0.006 at matched loss, with $r(\text{loss}, \text{accuracy}) = -0.956$ at that length. That recursion substitutes for depth is the premise of Mixture-of-Recurions (Bae et al., 2025).

The main caveat. No loss-matched analysis exists on Match-Query, so the loop’s gain there is associated with, not separated from, more reliable training.

Full report: report.pdf, Sections 9–10 and Appendices H–I (recursion on parity, Selective RoPE’s generator knob by knob, the forget gate, hierarchy compute, the correction ablation).

8 Claim 6: the two position mechanisms anti-compose unless the kernel can compensate

Two published mechanisms were crossed on their own tasks: MapFormer’s path integration (position is a cumulative sum of learned per-token increments) and PoPE’s decoupling (content in the magnitude, position in the phase, with a constant phase offset). Each replicates: on Dyck-2 a one-layer path-integrated model learns the bracket stack where a one-layer index model does not, and on Bach Chorales PoPE beats RoPE by

−0.032 NLL (5/5 seeds; paper −0.019). In distribution path integration adds nothing to PoPE (0/5 seeds better). Beyond the training context the ordering inverts and the naive combination collapses.

Table 7: Bach Chorales trained on 512-token crops, test NLL by position bucket (lower is better, 5 seeds). MapPoPE is the best arm in distribution and by far the worst outside it. Adding a per-token phase offset to its kernel — the freedom PoPE removes and rotating Q, K provides — fixes it.

arm	0–512 (trained)	512–1024	1024–2048
RoPE (index)	0.5512	0.8116	2.0587
PoPE (index)	0.5403	0.6749	1.5973
MapWM (path integration)	0.5382	0.7835	1.3969
MapPoPE (both)	0.5235	1.9536	4.6158
MapPoPE + per-token phase	0.5182	0.5629	0.6162

The mechanism is not the accumulator: its growth exponent is $\alpha \approx 1.00$ in both path-integrated arms, and they respond oppositely to compressing it at evaluation (−1.70 nats for MapPoPE, +1.44 for MapWM). It is that PoPE’s kernel has non-negative amplitudes and a constant phase, so when the accumulated argument leaves the range training calibrated, content cannot shift the peak to compensate. Two interventions confirm it: bounding the accumulator helps MapPoPE 5.7× more than MapWM, and restoring a per-token phase — accumulator untouched, parameter-matched inert twin flat — removes the collapse. The boundary is a double dissociation: the same phase is neutral to harmful where the accumulator is bounded (Dyck-2 −0.047 against its inert twin; torus 0.911 against 0.963 at $l = 2048$).

9 Limitations

- **Transfer across a change of structure is not measured.** Every evaluation redraws the observation map within one topology and action space. The TEM claim that motivates the work is about carrying structural knowledge across structures; nothing here tests it, and sample efficiency is not measured either.
- **One varying-offset task.** Claim 4 rests on k -back recency. MapWM at the larger budget was not run.
- **Out-of-distribution length.** Most accuracy effects are concentrated beyond the training length (rank, sign, the filter, the forget gate, PoPE’s magnitude), and no account of that concentration exists (full report, Section 11).
- **Loss-matched readings depend on the pool,** and they are uninformative where arm losses do not overlap.
- **Conditions established on different tasks.** The three conditions were shown on different tasks; no single task shows them binding together, and they are not claimed to be sufficient.
- **No language result.** The language experiments are underpowered or single-seed, and published work reports negatives for content-dependent phase on language.
- **Boundaries at $n = 3$, and Match-Query reproducibility.** The MiniWorld map-extent threshold, the 12-heading budget curve and several appendix results are exploratory; Match-Query does not reproduce across batches or same-seed retrains, so its paired interactions are not re-established.

10 Conclusion

On tasks that redraw the observation map, MapFormer’s transferable where is its accumulated, content-driven phase. It is the larger factor on raw accuracy on MapFormer’s own task, at training length under the paper’s recipe and at all three lengths under a converged one; the rotary encoding adds a smaller amount beyond the training length. The phase pays when the input makes each token’s displacement fixed, when increments are signed and well conditioned so that they cancel, and when training finds the solution. The first two conditions

were supported by single-operation interventions (the action record, the absolute value, the rank), though that rank acts through conditioning is descriptive; the third by constructing the solution, holding it, and measuring where from-scratch training finds it. A fully factorised where, MapEM’s shared kernel, represents the one varying-offset task measured exactly but is found per query token, and giving the kernel per-pair freedom changes both the accuracy and the mechanism. Filtering and gating the where add no measurable accuracy at training length on these tasks, and pooling’s only detectable gain is ceiling-level (its larger gains are directional); recursion does, in association with more reliable optimisation (on the torus its gain vanishes at matched loss; Match-Query was not loss-matched). What remains open is why so many mechanisms help only beyond the training length, and whether any of this transfers across a change of structure.

References

- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recurions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems*, 2025. Not in the local corpus. Identifier and venue from project notes; title and author first names not verified against the paper.
- Olga Golovneva, Tianlu Wang, Jason Weston, and Sainbayar Sukhbaatar. Contextual position encoding: Learning to count what’s important, 2024. Identifier from papers/txt/cope.txt; some repository results files cite 2405.11582 instead.
- Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations*, 2025.
- Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles, 2026.
- Sajad Movahedi, Timur Carstensen, Arshia Afzal, Frank Hutter, Antonio Orvieto, and Volkan Cevher. Selective rotary position embedding. In *International Conference on Learning Representations*, 2026.
- Alok Puranik. Using group theory to explore the space of positional encodings for attention. Jane Street Blog, April 2026. Blog post, no arXiv identifier; stored as papers/txt/puranik-janestreet.txt.
- Victor Rambaud, Salvador Mascarenhas, and Yair Lakretz. MapFormers: Self-supervised learning of cognitive maps with input-dependent positional encoding, 2025. v4, 10 May 2026, is the version cited; numbers marked v1 are from the November 2025 version.
- Yash Sarrof, Yana Veitsman, and Michael Hahn. The expressive capacity of state space models: A formal language perspective, 2024.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding, 2021.
- Ali Veisi, Delaram Fartoot, and Hamidreza Amirzadeh. Context-aware rotary position embedding, 2025.
- Nitin Vetcha. Towards infinite length extrapolation: A unified approach, 2026.
- James C. R. Whittington, Timothy H. Muller, Shirley Mark, Guifen Chen, Caswell Barry, Neil Burgess, and Timothy E. J. Behrens. The Tolman-Eichenbaum machine: Unifying space and relational memory through generalization in the hippocampal formation. *Cell*, 183:1249–1263, 2020. Not in the local corpus. Journal, volume and pages as cited in papers/txt/tale_two_algorithms.txt and papers/txt/kv_brain.txt; MapFormer cites the 2019 bioRxiv version.
- James C. R. Whittington, William Dorrell, Timothy E. J. Behrens, Surya Ganguli, and Mohamady El-Gaby. A tale of two algorithms: Structured slots explain prefrontal sequence memory and are unified with hippocampal cognitive maps. *Neuron*, 113(2):321–333, 2025. doi: 10.1016/j.neuron.2024.10.017. MapFormer’s reference [11]; stored as papers/txt/tale_two_algorithms.txt.

Yifan Zhang, Zixiang Chen, Yifeng Liu, Zhen Qin, Huizhuo Yuan, Kangping Xu, Yang Yuan, Quanquan Gu, and Andrew Chi-Chih Yao. Group representational position encoding. In *International Conference on Learning Representations*, 2026.